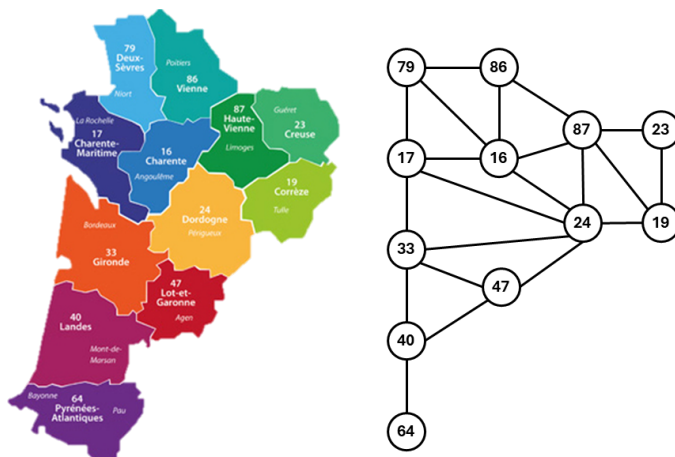


TP 13

Représentation des graphes – Sujet

Exercice d'application ★

On donne la carte de la région Nouvelle Aquitaine.



Des exemples de graphes ou dictionnaires sont présents ici : https://github.com/xpessoles/2025-2026_Enseignements/raw/main/PTSI_Informatique/TP/TP_13_ApplicationsGraphes/TP_13_ApplicationsGraphes.py.

FIGURE 1 – Carte et graphe de la région Nouvelle-Aquitaine

Dictionnaire

On souhaite créer un dictionnaire permettant de faire une correspondance entre le numéro des départements et leur nom.

Question 1 Implémenter une fonction `add_dpt(d:{}, dpt:int, nom:str) -> None` qui ajoute un département au dictionnaire `d`.

Exemple – Tests

```
1 >>> dico = {}
2 >>> add_dpt(dico, 64, "Pyrénées-Atlantiques")
3 >>> print(dico)
4 {64: "Pyrénées-Atlantiques"}
```

Dictionnaire d'adjacence

On souhaite modéliser la carte par un dictionnaire d'adjacence où les sommets sont des numéros de départements et les valeurs sont la liste des départements limitrophes.

Question 2 Donner l'instruction permettant de créer un graphe appelé `aqui` avec les Pyrénées-Atlantiques (64) et les Landes uniquement (40).

Question 3 Implémenter la fonction `existe_d(G:{}, dpt:int) -> bool` qui renvoie `True` si le département est dans le graphe, `False` sinon. Tester votre fonction.

Question 4 Implémenter la fonction `ajoute_dpt_d(G:{}, dpt:int) -> None` qui ajoute un département au graphe `G`. Tester votre fonction.

Question 5 Implémenter la fonction `sont_voisins_d(G:{}, dpt1:int, dpt2:int) -> bool` qui renvoie `True` si les deux départements sont voisins, `False` sinon. Tester votre fonction.

Question 6 Implémenter la fonction `ajoute_une_frontiere_d(G:{}, s1: int, s2: int) -> None` qui ajoute une arête `G` entre deux départements limitrophes. Tester votre fonction.

Question 7 Implémenter la fonction `ajoute_dpt_voisins_d(G:{}, dpt:int, voisins : [int]) -> None` qui ajoute un département au graphe `G`. `dpt` désigne le département à ajouter et `voisins` la liste des départements voisins. Tester votre fonction.

Question 8 Implémenter la fonction `voisins_d(G:{}, dpt:int) -> [int]` qui renvoie la liste des départements voisins de `dpt`. Tester votre fonction.

Question 9 Implémenter la fonction `suppr_dpt_d(G:{}, dpt:int) -> None` qui supprime un département de la région. Tester votre fonction.

Question 10 Implémenter la fonction `frontieres_d(G:{}) -> [()]` qui renvoie la listes des arêtes du graphe. Une arête sera une tuple (couple) de deux départements limitrophes. On supprimera les doublons. Tester votre fonction.

Question 11 Implémenter la fonction `degre_d(G:{}, dpt:int) -> int` qui donne le degré d'un sommet. Tester votre fonction.

Question 12 Implémenter la fonction `degre_max_d(G:{}) -> int` qui donne le degré maximum du graphe. Tester votre fonction.

Question 13 Implémenter la fonction `degre_max_l_d(G:{}) -> [int]` qui renvoie la liste des sommets de plus haut degré. Tester votre fonction.

Question 14 Implémenter la fonction `degre_max_dpt_d(G:{}, d:{}) -> [str]` prend en argument le graphe de la région ainsi que le dictionnaire des départements qui renvoie le nom du (ou des) départements ayant le plus de départements limitrophes. Tester votre fonction.

Déplacement d'un cavalier sur un échiquier

Un cavalier se déplace, lorsque c'est possible, de 2 cases dans une direction verticale ou horizontale, et de 1 case dans l'autre direction (le trajet dessine une figure en L).

Dans un premier temps, les cases de l'échiquier sont représentées par des tuples : le couple (i, j) désigne la case d'abscisse i et d'ordonnée j . Un échiquier possède 8 colonnes et 8 lignes, donc i et j seront compris entre 0 et 7.

Question 15 Écrire une fonction `estDansEch(i:int, j:int) -> bool` qui renvoie `True` si (i, j) correspond à une case valide de l'échiquier et `False` sinon.

Question 16 Écrire une fonction `mvtsPossibles(i:int, j:int)->list` qui renvoie la liste des cases où le cavalier peut se déplacer à partir de la case (i, j) à l'ordre près.

Question 17 Vérifier que :

- ▶ `mvtsPossibles(0,0)` renvoie $[(1, 2), (2, 1)]$,
- ▶ `mvtsPossibles(3,5)` renvoie bien $[(1, 4), (1, 6), (2, 3), (2, 7), (4, 3), (4, 7), (5, 4), (5, 6)]$,
- ▶ `mvtsPossibles(7,7)` renvoie bien $[(5, 6), (6, 5)]$.

Tous ces résultats sont à l'ordre près.

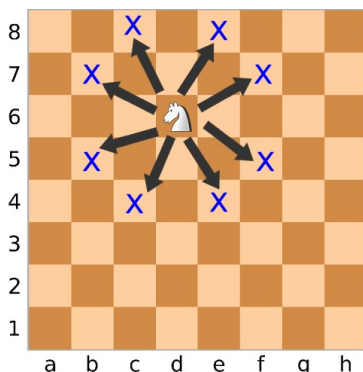


FIGURE 2 – Illustration du mouvement d'un cavalier sur un échiquier

Question 18 Créer un graphe `G` sous la forme d'un dictionnaire d'adjacence avec pour sommets les différentes cases de l'échiquier et les arêtes qui correspondent à un mouvement possible du cavalier.

Question 19 Vérifiez que vous avez un graphe avec 64 sommets et 168 arêtes.

Le codage des cases d'échiquier se fait classiquement par une chaîne de caractères comprenant : une lettre minuscule pour l'abscisse (de `a` à `h`) et un chiffre pour l'ordonnée (de 1 à 8). La case en bas à gauche de l'échiquier est donc de code `'a1'`.

Remarque

`ord(c)` retourne le codage Unicode correspondant au caractère `c` et `chr(n)` renvoie le caractère dont le codage Unicode est `n`.

Question 20 Écrire une fonction `codage(i:int, j:int)->str` qui renvoie le code correspondant à la case `(i, j)`.

Question 21 Créer un dictionnaire d'adjacence `G2` comme défini précédemment avec les cases maintenant nommées d'après leur code.

Question 22 Vérifier que, à l'ordre près :

- ▶ `G2['a1']` renvoie `['b3', 'c2']`,
- ▶ `G2['d6']` renvoie bien `['b5', 'b7', 'c4', 'c8', 'e4', 'e8', 'f5', 'f7']`,
- ▶ `G2['h8']` renvoie bien `['f7', 'g6']`.

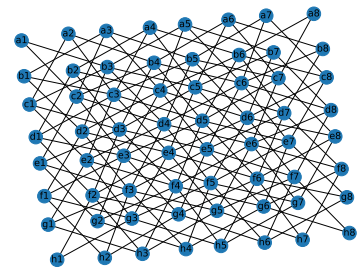


FIGURE 3 – Représentation graphique du graphe `G2`